

Knapsack Problem:

The Knapsack Problem is an optimization problem where we have a set of items, each with a weight and value. We need to select items to put into a knapsack with a limited capacity so that the total value is maximized without exceeding the capacity.

ALGORITHM:

- Create a DP table $DP[i][w]$, where i represents items and w represents capacity.
- For each item, check whether its weight can fit into the current capacity.
- If the item does not fit, exclude it.
- If it fits, choose the maximum of:
 - Value without including the item.
 - Value obtained by including the item.
- The final table value gives the maximum possible profit.

Case	Time Complexity	Space Complexity
Best Case	$O(n \times W)$	$O(n \times W)$
Average Case	$O(n \times W)$	$O(n \times W)$
Worst Case	$O(n \times W)$	$O(n \times W)$

OUTPUT:

```
Enter number of items: 4
Enter weights of items:
4
5
3
8
Enter values of items:
2
3
4
5
Enter knapsack capacity: 5
Maximum value = 4

...Program finished with exit code 0
Press ENTER to exit console.
```

Conclusion:

The Knapsack Problem demonstrates how **Dynamic Programming** can be used to solve optimization problems efficiently. The 0/1 Knapsack algorithm finds the maximum value that can be obtained without exceeding the given capacity.